

OSSP-SKILL-WEEK7

Session 13

TASK1: Implement a program to parse input redirection, open files, redirect stdin, handle missing files, restore streams, and test file inputs.

SOURCE CODE:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char *argv[])
{
    int fd;
    int saved_stdin;
    char buffer[200];
    ssize_t bytes;

    printf("Session 13 - Task 1: Input Redirection\n");
    printf("-----\n");

    if (argc != 2)
    {
        printf("Usage: %s <input_file>\n", argv[0]);
        return 1;
    }

    fd = open(argv[1], O_RDONLY);

    if (fd == -1)
    {
        perror("Error opening input file");
        return 1;
    }

    // Save original stdin
    saved_stdin = dup(STDIN_FILENO);

    // Redirect stdin to file
    if (dup2(fd, STDIN_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Input redirected from: %s\n\n", argv[1]);
    printf("Data read from file:\n");

    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0)
    {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    if (bytes == -1)
        perror("Error reading file");

    // Restore original stdin
    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);

    printf("\n\nOriginal stdin restored successfully.\n");

    return 0;
}
```

OUTPUT:

```

Long pipeline execution completed successfully.
root@LAPTOP-1COJKKG6:~# nano input.txt
root@LAPTOP-1COJKKG6:~# nano session13_task1.c
root@LAPTOP-1COJKKG6:~# gcc session13_task1.c -o session13_task1
root@LAPTOP-1COJKKG6:~# ./session13_task1 input.txt
Session 13 - Task 1: Input Redirection
-----

Input redirected from: input.txt

Data read from file:
Hello, this is input redirection.
This data is coming from a file.

Original stdin restored successfully.
root@LAPTOP-1COJKKG6:~# |

```

OBSERVATION: I learned how input redirection works in Linux using `open()`, `dup()`, and `dup2()`. I understood how a file can be connected to standard input, how data can be read from the file, how missing files are handled, and how the original stdin can be restored.

TASK2: Implement a program to parse output redirection, create output files, redirect stdout, handle permissions, verify output data, and test redirection.

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char *argv[])
{
    int fd;
    int saved_stdout;

    if (argc != 2)
    {
        printf("Usage: %s <output_file>\n", argv[0]);
        return 1;
    }

    printf("Session 13 - Task 2: Output Redirection\n");
    printf("-----\n");

    // Save original stdout
    saved_stdout = dup(STDOUT_FILENO);

    // Create/open output file
    fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {

```

```

GNU nano 7.2

if (fd == -1)
{
    perror("Error opening output file");
    close(saved_stdout);
    return 1;
}

// Redirect stdout to file
if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdout);
    return 1;
}

close(fd);

printf("Output redirection successful.\n");
printf("This data has been written to the output file.\n");
printf("File: %s\n", argv[1]);

fflush(stdout);

// Restore original stdout
dup2(saved_stdout, STDOUT_FILENO);
close(saved_stdout);

printf("Original stdout restored successfully.\n");

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session13_task2.c
root@LAPTOP-1COJKKG6:~# gcc session13_task2.c -o session13_task2
root@LAPTOP-1COJKKG6:~# ./session13_task2 output.txt
Session 13 - Task 2: Output Redirection
-----
Original stdout restored successfully.
root@LAPTOP-1COJKKG6:~# cat output.txt
Output redirection successful.
This data has been written to the output file.
File: output.txt
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how output redirection works by connecting standard output to an output file using `dup2()`. I understood how files are created with appropriate permissions, how output data is written to the file, and how the original stdout stream can be restored.

Session-14

Task1: Implement append redirection. The program should parse the append operator, open files in append mode, redirect streams, preserve existing data, validate appends, and test different scenarios.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char *argv[])
{
    int fd;
    int saved_stdout;

    if (argc != 2)
    {
        printf("Usage: %s <output_file>\n", argv[0]);
        return 1;
    }

    printf("Session 14 - Task 1: Append Redirection\n");
    printf("-----\n");

    // Save original stdout
    saved_stdout = dup(STDOUT_FILENO);

    // Open file in append mode
    fd = open(argv[1], O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1)
    {
        perror("Error opening output file");
        close(saved_stdout);
        return 1;
    }

    // Redirect stdout to the file
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
    }
}
```

```

if (fd == -1)
{
    perror("Error opening output file");
    close(saved_stdout);
    return 1;
}

// Redirect stdout to the file
if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdout);
    return 1;
}

close(fd);

printf("New data appended to the file.\n");
printf("Existing data is preserved.\n");
printf("Append operation completed successfully.\n");

fflush(stdout);

// Restore original stdout
dup2(saved_stdout, STDOUT_FILENO);
close(saved_stdout);

printf("Original stdout restored successfully.\n");

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session14_task1.c
root@LAPTOP-1COJKKG6:~# gcc session14_task1.c -o session14_task1
root@LAPTOP-1COJKKG6:~# echo "This is existing data." > append.txt
root@LAPTOP-1COJKKG6:~# cat append.txt
This is existing data.
root@LAPTOP-1COJKKG6:~# ./session14_task1 append.txt
Session 14 - Task 1: Append Redirection
-----
Original stdout restored successfully.
root@LAPTOP-1COJKKG6:~# cat append.txt
This is existing data.
New data appended to the file.
Existing data is preserved.
Append operation completed successfully.
root@LAPTOP-1COJKKG6:~# ./session14_task1 append.txt
Session 14 - Task 1: Append Redirection
-----
Original stdout restored successfully.
root@LAPTOP-1COJKKG6:~# cat append.txt
This is existing data.
New data appended to the file.
Existing data is preserved.
Append operation completed successfully.
New data appended to the file.
Existing data is preserved.
Append operation completed successfully.
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to implement **append redirection** using the `O_APPEND` flag with `open()`. I understood how `dup2()` redirects standard output to a file, how existing file data is preserved, and how new data is added at the end of the file. I also learned how to restore the original stdout after redirection.

Task2: Implement stderr redirection. The program should parse stderr redirection, redirect error streams, capture error output, verify file contents, handle failures, and test error cases.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

int main(int argc, char *argv[])
{
    int fd;
    int saved_stderr;

    if (argc != 2)
    {
        printf("Usage: %s <error_file>\n", argv[0]);
        return 1;
    }

    printf("Session 14 - Task 2: stderr Redirection\n");
    printf("-----\n");

    // Save original stderr
    saved_stderr = dup(STDERR_FILENO);

    // Open error file
    fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("Error opening error file");
        close(saved_stderr);
        return 1;
    }

    // Redirect stderr to the file
    if (dup2(fd, STDERR_FILENO) == -1)
    {
        perror("dup2");
    }
}
```

```

// Redirect stderr to the file
if (dup2(fd, STDERR_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stderr);
    return 1;
}

close(fd);

// These error messages go into the file
fprintf(stderr, "This is an error message.\n");
fprintf(stderr, "stderr redirection is working successfully.\n");
fprintf(stderr, "Error output has been captured in the file.\n");

fflush(stderr);

// Restore original stderr
dup2(saved_stderr, STDERR_FILENO);
close(saved_stderr);

printf("Original stderr restored successfully.\n");

return 0;
}

```

Output:

```

Append operation completed successfully.
root@LAPTOP-1COJKKG6:~# nano session14_task2.c
root@LAPTOP-1COJKKG6:~# gcc session14_task2.c -o session14_task2
root@LAPTOP-1COJKKG6:~# ./session14_task2 error.txt
Session 14 - Task 2: stderr Redirection
-----
Original stderr restored successfully.
root@LAPTOP-1COJKKG6:~# cat error.txt
This is an error message.
stderr redirection is working successfully.
Error output has been captured in the file.
root@LAPTOP-1COJKKG6:~# ./session14_task2 /invalid/path/error.txt
Session 14 - Task 2: stderr Redirection
-----
Error opening error file: No such file or directory
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how to redirect the stderr stream using `dup2()` and file descriptors. I understood how error messages can be captured in a file, how to verify the redirected error output, handle file-opening failures, and restore the original stderr stream.